UNIVERSIDAD ABIERTA Y A DISTANCIA DE MÉXICO

CIENCIAS EXACTAS, INGENIERÍA Y TECNOLOGÍA

MATEMÁTICAS

ANÁLISIS NUMÉRICO I

Nombre del estudiante: Alexi Uriel Cabrera Mayoral

Grupo: MT-MANU1-2601-B2-001

Actividad 2. Modelación matemática

Fecha de entrega: 19/04/2026

Introducción

La modelación matemática es fundamental para presentar fenómenos del mundo real con expresiones matemáticas que describen su comportamiento. A través de un modelo matemático, es posible analizar, predecir y comprender cómo evoluciona un sistema bajo distintas condiciones, sin necesidad de realizar experimentos físicos caros o peligrosos. Estos modelos pueden simples como una ecuación lineal o tan complejos como un sistema de ecuaciones diferenciales parciales, dependiendo de la naturaleza del fenómeno que se desea estudiar.
Una clásica modelación matemática es el tiro parabólico porque describe el movimiento de un proyectil lanzado con una velocidad inicial bajo la influencia de la gravedad. En este caso, la altura del proyectil en función del tiempo puede expresarse mediante una ecuación cuadrática, lo que permite calcular la trayectoria seguida por el objeto.
El uso de software como Octave, facilita la evaluación de estos modelos para múltiples valores de entrada, pudiendo visualizar y analizar los resultados.

Desarrollo

Implementa en Octave el modelo de tiro parabólico, considerando lo siguiente:
La ecuación que describe el tiro parabólico desde el origen es:

$$h(t) = -\frac{g}{3}t^2 + vt$$

donde g = 9.8 $m/s2$ y v es la velocidad inicial del cuerpo proyectado.
La tarea es crear una función en Octave que recibe dos parámetros y regresa un escalar que representará la altura del proyectil en cada tiempo t. Deberás mostrar la trayectoria h(t) seguida para los siguientes valores de v y vectores t.

V	Vectores de T
10	(0,10,100)
25	(0,20,250)
100	(0,50,300)

Para definir la serie de valores donde aplicarás tu función, es decir, todos los valores t tienes que hacer uso de la función linspace de la siguiente manera
`>>> t = linspace(0,10,100)`
Que quiere decir que t es un vector con valores reales entre 0 y 10 con 100 puntos distribuidos equidistantemente.
Llama a tu función “ `tiroP` ”

Recuerda por cada función debes crear un archivo que se llame igual que la función, pero con extensión .m y en una carpeta de tu elección. Guarda tus archivos .m generados en Octave con la siguiente nomenclatura:

- tiroP.m para el archivo que contiene a la función tiroP
- ejemplo.m para el archivo donde calculaste la función tiroP con los valores $v1 = 10$ y $t1 = (0, 10, 100), v2 = 25$ y $t2 = (0, 20, 250)$ y $v3 = 100 t3 = (0, 50, 300)$.
  Recuerda agregar el comando que te permita imprimir todos los resultados.

Tip: Para definir una función, llamada funcX, de n parámetros en Octave la sintaxis es la siguiente
`function y = funcX(n1,n2,…,nk)`
Concretamente una función de dos parámetros x, y se define como
`function y = funcX(x,y)`
Tip 2: Octave permite el uso de operaciones puntuales, es decir, extiende la aplicación de la suma, producto y división (entre otras) a vectores, lo que significa que puedes sumar dos vectores de manera implícita de la siguiente forma. Supongamos que X e Y son vectores,
`>>> Z = X*.Y`
El operador punto (.) le indica a Octave que tiene que multiplicar el elemento i-ésimo de X por el elemento i-ésimo de Y y construir al mismo tiempo el vector Z de tal forma que `Z(i) = X(i) * Y(i)`
Si tenemos la función funcM que es una función que recibe un solo parámetro y regresa un escalar, entonces podemos aplicarla a todo el vector X de la siguiente forma
`Z = funcM(X)`

Toma y agrega 3 capturas de pantalla de tu trabajo en Octave: una captura de pantalla de la función que programaste en Octave (tiroP.m), una del archivo donde muestres los valores v y t y la llamada a la función (ejemplo.m), y una de la pantalla de comandos con los resultados obtenidos.

Resultados

Función `tiroP.m`, donde ocupamos el tip 2 para escribir la función h:

```
h = -(g/3) .* t.^2 + v .* t
```

```
function h = tiroP(v, t)
  g = 9.8;
  h = -(g/3) .* t.^2 + v .* t;
endfunction
```

Casos planteados para `ejemplo.m`. Llamamos a la función, dándole los argumentos correspondientes.

tiroP.m ejemplo.m

```
% Caso 1: v=10, t entre 0 y 10 con 100 puntos
v1 = 10;
t1 = linspace(0, 10, 100);
h1 = tiroP(v1, t1);
disp('Resultados para v=10, t=(0,10,100):');
disp(h1);

% Caso 2: v=25, t entre 0 y 20 con 250 puntos
v2 = 25;
t2 = linspace(0, 20, 250);
h2 = tiroP(v2, t2);
disp('Resultados para v=25, t=(0,20,250):');
disp(h2);

% Caso 3: v=100, t entre 0 y 50 con 300 puntos
v3 = 100;
t3 = linspace(0, 50, 300);
h3 = tiroP(v3, t3);
disp('Resultados para v=100, t=(0,50,300):');
disp(h3);
```

Ejecución de `ejemplo.m` donde obtenemos correctamente los resultados, esto ultimo porque al hacer

una interpretación, se aprecia como sube, alcanza una altura máxima y luego baja.

```
>> cd MANU1/
>> cd Unidad1/
>> ejemplo
Resultados para v=10, t=(0,10,100):
 Columns 1 through 6:

         0    0.9768    1.8869    2.7303    3.5071    4.2173

 Columns 7 through 12:

    4.8607    5.4375    5.9477    6.3912    6.7680    7.0782

 Columns 13 through 18:

    7.3217    7.4986    7.6087    7.6523    7.6292    7.5394

 Columns 19 through 24:

    7.3829    7.1598    6.8700    6.5136    6.0905    5.6008

 Columns 25 through 30:

    5.0444    4.4213    3.7316    2.9752    2.1522    1.2625

 Columns 31 through 36:

    0.3061   -0.7169   -1.8066   -2.9630   -4.1860   -5.4756

 Columns 37 through 42:

   -6.8320   -8.2549   -9.7446  -11.3009  -12.9239  -14.6135

 Columns 43 through 48:

  -16.3698  -18.1927  -20.0823  -22.0386  -24.0615  -26.1511

 Columns 49 through 54:

  -28.3073  -30.5302  -32.8198  -35.1760  -37.5989  -40.0884

 Columns 55 through 60:

  -42.6446  -45.2675  -47.9570  -50.7132  -53.5360  -56.4255

 Columns 61 through 66:

  -59.3817  -62.4045  -65.4940  -68.6501  -71.8729  -75.1624

 Columns 67 through 72:

  -78.5185  -81.9413  -85.4307  -88.9868  -92.6096  -96.2990

 Columns 73 through 78:

 -100.0551 -103.8778 -107.7672 -111.7233 -115.7460 -119.8354
```

```
 Columns 79 through 84:

  -123.9914  -128.2141  -132.5035  -136.8595  -141.2822  -145.7715

 Columns 85 through 90:

  -150.3275  -154.9502  -159.6395  -164.3955  -169.2181  -174.1074

 Columns 91 through 96:

  -179.0634  -184.0860  -189.1753  -194.3312  -199.5538  -204.8430

 Columns 97 through 100:

  -210.1990  -215.6215  -221.1108  -226.6667
Resultados para v=25, t=(0,20,250):
 Columns 1 through 6:

         0     1.9870     3.9318     5.8344     7.6949     9.5133

 Columns 7 through 12:

    11.2895    13.0236    14.7155    16.3652    17.9728    19.5383

 Columns 13 through 18:

    21.0616    22.5428    23.9818    25.3786    26.7333    28.0459
```

Conclusión

Realizar esta práctica permitió comprender concretamente como la modelación matemática se puede implementar computacionalmente simulando fenómenos físicos reales.
Usando Octave se definió una representación de la función de la altura $h(t)$ de un proyectil en el tiempo y aplicarla a distintos vectores de valores observando la variación de la trayectoria desde la velocidad inicial al revisar los valores numéricos.

Se usaron operaciones puntuales en Octave siendo este clave al tener la utilidad de aplicar funciones matemáticas a vectores o casos de forma directa evitando la necesidad de ciclos explícitos. Esto refleja una de las grandes ventajas del software de cómputo científico: la capacidad de manejar grandes volúmenes de datos con pocas líneas de código.

Referencias

- Index of /gnu/octave/windows/. (s. f.). https://mirror.team-cymru.com/gnu/octave/windows/
- Octave. (s. f.). https://wiki.octave.org/GNU_Octave_Wiki